

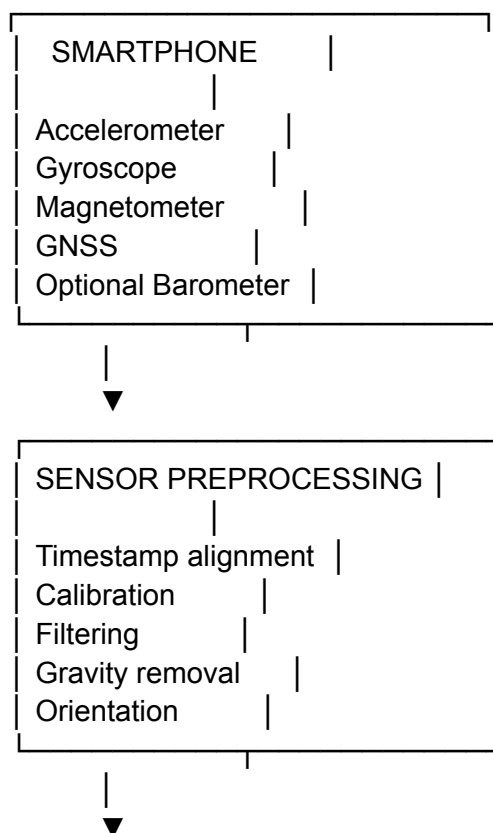
Intelligent Dead Reckoning — Optimized End-to-End Prototype Plan

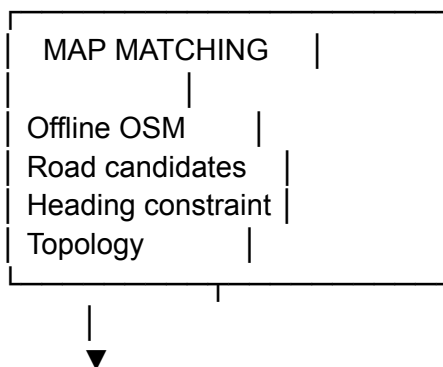
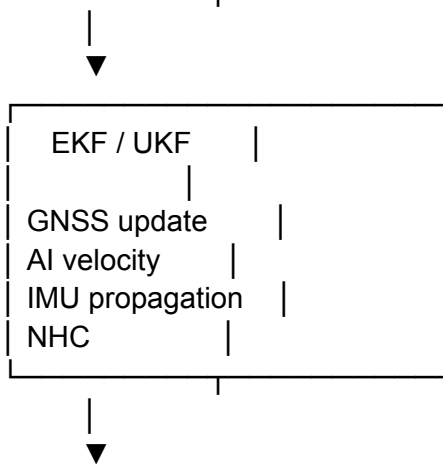
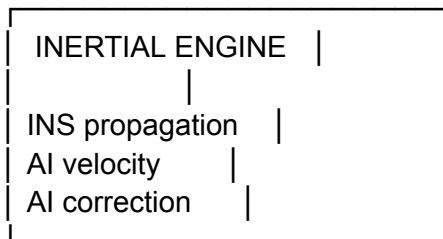
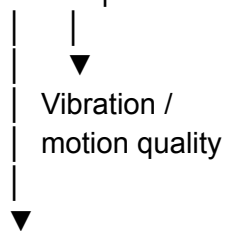
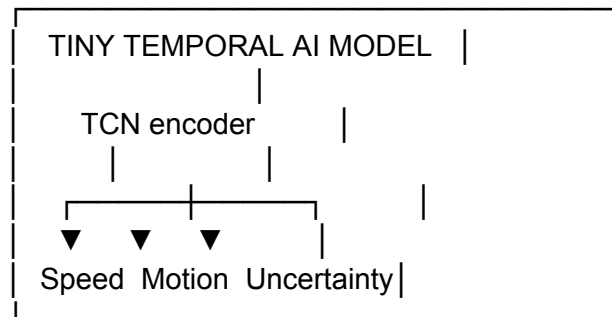
Goal: Build a working smartphone-only prototype first: phone IMU → lightweight ML → dead reckoning/fusion → offline map → mobile Chrome web app.
Raspberry Pi / external GPS / FOG / external IMU are **not part of the first implementation**. They remain future interfaces for the same core engine.

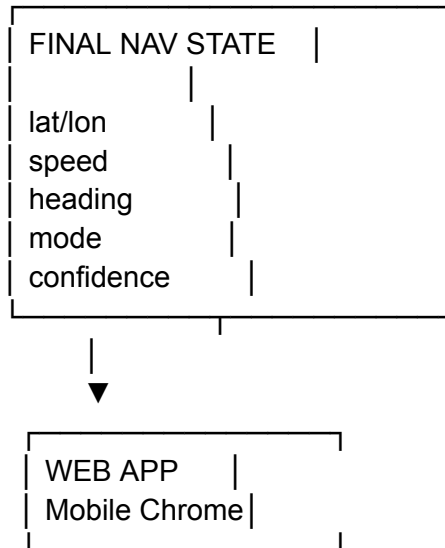
The architecture below is deliberately designed so that the **internal hackathon prototype can be built quickly**, while the same components can evolve into the final SIH system.

0. Executive Architecture

The system should ultimately look like this:







The critical design principle is:

AI estimates what classical inertial navigation is bad at; physics/filtering determines how that information becomes position.

PART I — SMARTPHONE SENSOR PIPELINE

1. Sensor Acquisition

Your first team member is handling this, but define the interface **before they implement it**.

The core engine should not know anything about Android APIs.

Define a common structure:

SensorFrame

timestamp_ns

accelerometer:

ax
ay
az

gyroscope:

gx
gy
gz

magnetometer:

mx
my
mz

GNSS:

latitude
longitude
altitude
speed
bearing
accuracy
timestamp

optional:

barometric_altitude

Then your pipeline consumes:

SensorFrame

regardless of where it came from.

Later:

Android → SensorFrame
Raspberry Pi → SensorFrame
FOG IMU → SensorFrame
IO-VNBD → SensorFrame

This is **extremely important** for satisfying the final PS requirement.

2. Android Sensors

For the first prototype:

Required

- Accelerometer
- Gyroscope
- GNSS

Useful

- Magnetometer

Optional

- Barometer

Don't make barometer or magnetometer a hard dependency.

3. Sampling Rate

For a real phone:

Target:

Accelerometer: 50–100 Hz

Gyroscope: 50–100 Hz

Magnetometer: 20–50 Hz

GNSS: 1–10 Hz

The actual rate depends on the phone.

Your software must therefore **timestamp every sample**.

Never assume:

sample arrives every exactly 10 ms

Instead:

$dt = \text{timestamp}[i] - \text{timestamp}[i-1]$

4. Timestamp Synchronization

This is one of the highest-priority parts.

You may receive:

Accelerometer:

10:00:00.000

10:00:00.011

10:00:00.020

Gyroscope:

10:00:00.003

10:00:00.013

10:00:00.023

GNSS:

10:00:00.000

10:00:01.000

Everything needs to be placed on a common timeline.

Use:

monotonic sensor timestamps

rather than wall-clock time where possible.

5. Resampling

Create a common IMU timeline.

For example:

100 Hz:

0.00

0.01

0.02

...

1.99

2.00

Interpolate the lower-frequency channels appropriately.

Do **not** blindly interpolate raw GNSS positions at 100 Hz and treat those as ground truth.

GNSS remains a low-frequency measurement.

6. Coordinate Frames

Define this once and never change it.

Use vehicle coordinates:

X = forward

Y = left/right

Z = up

or:

X = forward

Y = left

Z = up

Then document it everywhere.

7. Phone Orientation

Phone coordinates are not vehicle coordinates.

You need:

$\mathbf{R}_{\{\text{phone} \rightarrow \text{vehicle}\}}$

such that:

$\mathbf{a}_v = \mathbf{R}_{\{pv\}} \mathbf{a}_p$

For the first version, get orientation from:

- gravity
- gyroscope
- Android rotation vector if available

Then estimate vehicle-forward direction from initial GNSS motion.

8. Automatic Initial Alignment

During the initial GNSS-available period:

Step 1

Estimate vertical using gravity.

Step 2

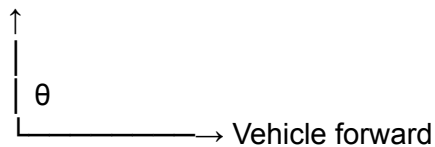
Estimate phone roll/pitch.

Step 3

Use GNSS course + vehicle movement to estimate yaw.

For example:

Phone forward



Estimate:

$$\theta = \text{wrap}(\psi_{\text{GNSS}} - \psi_{\text{phone}})$$

while the vehicle is moving sufficiently fast.

Don't try to estimate heading while stationary.

9. Calibration

During the first few seconds:

Accelerometer

Estimate:

- bias
- mean
- noise variance
- gravity magnitude

Gyroscope

Estimate:

- zero-rate bias
- noise variance

A simple stationary calibration:

$$\mathbf{b}_a = \text{mean}(\mathbf{a})$$

relative to expected gravity.

For gyro:

$$\mathbf{b}_g = \text{mean}(\boldsymbol{\omega})$$

10. Filtering

Don't immediately throw a neural network at raw signals.

Pipeline:

RAW IMU

↓

outlier rejection

↓

bias correction

↓

orientation transformation

↓

light filtering

↓
AI

Use simple filters first.

Recommended

- median/Hampel outlier filter
- low-pass filter for acceleration
- high-frequency branch for vibration detection

Don't over-smooth acceleration because you will destroy actual braking/acceleration signals.

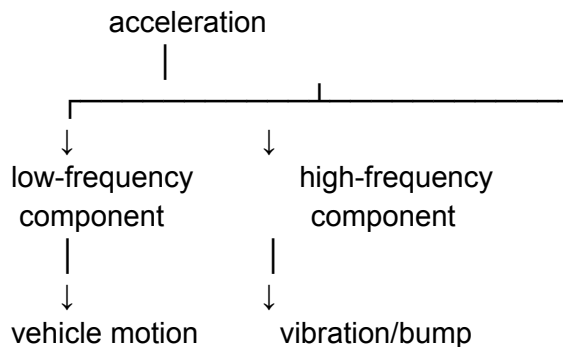
11. Split the Accelerometer Signal

This is useful.

Think of:

$a = a_{\text{vehicle}} + a_{\text{vibration}} + \text{noise}$

Create:



The high-frequency component becomes an input to the motion-quality model.

12. Sensor Quality Features

Generate inexpensive derived features:

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad |\omega| = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2} \quad j = \frac{da}{dt}$$

where j is jerk.

The ML input becomes approximately:

a_x a_y a_z

g_x g_y g_z

$|a|$

$|gyro|$

jerk

Nine channels.

13. Sensor Pipeline Output

Your sensor module should ultimately emit:

NormalizedFrame:

timestamp

vehicle_ax

vehicle_ay

vehicle_az

vehicle_gx

vehicle_gy

vehicle_gz

accel_magnitude

gyro_magnitude

jerk

orientation

orientation_confidence

gnss_position

gnss_velocity

gnss_accuracy

gnss_quality

Now Part II doesn't care how the phone obtained the data.

PART II — CORE MODEL + NAVIGATION PIPELINE

This is the part I would own most aggressively.

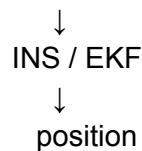
14. First Principle

Don't build:

IMU → Neural Network → GPS coordinates

Build:

IMU → Neural Network → learned motion



This follows the broad successful direction of learned inertial odometry such as RoNIN and TLIO, where learned inertial motion/velocity is combined with classical state estimation rather than asking a neural network to directly produce an unconstrained global trajectory.

[ScienceDirect](#)

15. Dataset: IO-VNBD

IO-VNBD is particularly appropriate because it contains smartphone IMU/GNSS data collected at 10 Hz alongside vehicle-side sensing and diverse driving conditions. The smartphone collection covers roughly 58 hours / 4,400 km. [ScienceDirect](#)

Use it for:

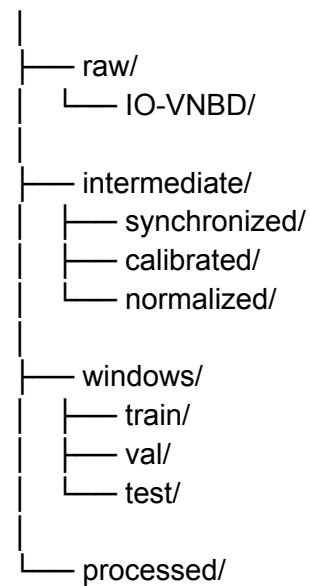
1. Baseline.
2. Model training.

3. Model validation.
 4. GNSS outage simulation.
 5. Ablation studies.
-

16. Dataset Directory

Set up:

data/



Don't modify raw data.

17. Dataset Preprocessing

Pipeline:

IO-VNBD



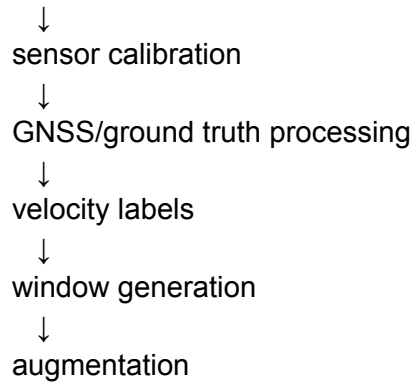
parse



timestamp synchronization



coordinate transformation



18. Ground Truth

For each time t , obtain:

position
velocity
heading

where possible.

Velocity can be derived from reference trajectory.

The vehicle-side data can also be used as training supervision; importantly, **wheel speed must not be fed into the deployment model**.

Use it as a teacher/ground-truth signal.

19. Teacher–Student Option

This is an excellent optimization if time permits.

Teacher

GNSS
+
vehicle reference sensors

Student

smartphone IMU only

Train:

$\backslash \text{Student(IMU)} \backslash \text{approx Teacher(state)} \backslash$

Deployment:

IMU only

This directly aligns with the PS requirement.

20. Training Windows

Start with:

$\backslash 2 \backslash \text{seconds} \backslash$

At IO-VNBD's 10 Hz:

$\backslash 20 \backslash \text{times } C \backslash$

At your later 100 Hz phone data:

$\backslash 200 \backslash \text{times } C \backslash$

where $\backslash(C\backslash)$ is the number of input channels.

Benchmark:

1 s

2 s

4 s

I expect 2–4 s to be the sweet spot.

21. Training Labels

Primary:

v_f

Forward velocity.

Secondary:

v_y

lateral velocity.

Motion class:

stationary
accelerating
cruising
braking
turning
bump

Optional:

velocity uncertainty

22. Model #1 — Baseline

Start with an MLP.

features

↓

Dense 64

↓

Dense 32

↓

speed

Purpose:

not deployment.

It tells you whether your preprocessing and labels work.

If MLP performs terribly, don't blame the TCN.

23. Model #2 — 1D CNN

Then:

IMU sequence



Conv1D



Conv1D



Pooling



Dense



speed

This establishes the temporal baseline.

24. Model #3 — GRU

Then:

IMU



GRU 32



Dense



speed

Use it as a comparison.

25. Model #4 — Recommended Final

Tiny TCN

Architecture:

Input
 $T \times 9$



Conv1D
32 channels
kernel 5



ReLU



Dilated Conv1D
32 channels
dilation 2



Residual connection



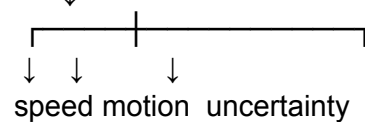
Dilated Conv1D
64 channels
dilation 4



Global Average Pooling



Dense 64



This is the model I would initially target.

26. Why TCN

Compared with a Transformer:

- substantially simpler
- smaller
- lower latency
- easy to quantize
- excellent temporal receptive field

Compared with LSTM/GRU:

- easier parallel training
- simpler deployment
- no recurrent hidden-state management
- naturally handles fixed windows

A ResNet-style 1D model is also a very good benchmark.

27. Model Outputs

Make the model output:

$[\hat{v}_f, \hat{v}_y, \hat{c}, \hat{\sigma}_v]$

where:

- $\hat{v}_f$: forward velocity
- $\hat{v}_y$: lateral velocity
- $\hat{c}$: motion class probabilities
- $\hat{\sigma}_v$: velocity uncertainty

This gives you much more useful information than only speed.

28. Residual Learning

An even better variant:

Instead of predicting:

$$\hat{v}$$

predict:

$$\Delta v$$

where:

$$v_{\text{corrected}} = v_{\text{physics}} + \Delta v$$

The network learns the **error in the classical estimate**.

This is inspired by physics-informed/residual inertial approaches such as recent PiDR work.
[ScienceDirect](#)

29. Physics-Informed Training

You don't need a huge textbook PINN.

Use physics as **additional loss terms**.

Primary:

$$L_{\text{data}} = \text{Huber}(\hat{v}, v_{\text{GT}})$$

Physics:

$$L_{\text{physics}} = \left| \hat{v}_{t+1} - (\hat{v}_t + a_t \Delta t) \right|^2$$

Vehicle constraint:

$$L_{\text{NHC}} = |\hat{v}_y|^2$$

Smoothness:

$$L_{\text{smooth}} = |\hat{v}_{t+1} - \hat{v}_t|$$

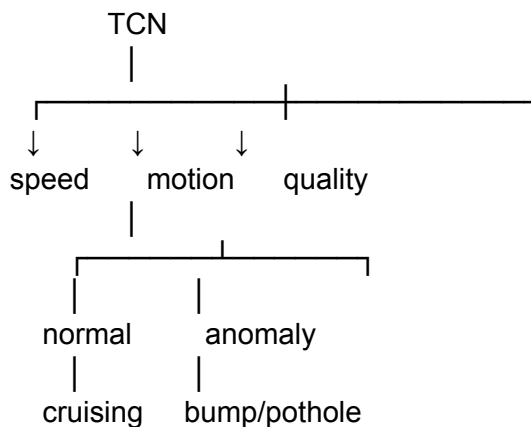
Then:

$$\boxed{L = L_{\text{data}} + \lambda_p L_{\text{physics}} + \lambda_n L_{\text{NHC}} + \lambda_s L_{\text{smooth}}}$$

This gives you the **PINN/physics-informed benefit** without turning the entire project into a **giant PINN research project**.

30. Motion Classification

Train a small classification head:



The classification output affects the filter.

31. Adaptive Trust

This is where the system becomes intelligent.

Suppose:

motion = CRUISING

confidence = high

Then:

$\downarrow R_{AI}$

and the filter trusts AI velocity.

But:

motion = POTHOLE

confidence = low

Then:

$$R_{AI} \uparrow$$

and the filter largely ignores that measurement.

This is much more useful than merely displaying "AI detected pothole."

32. Uncertainty Head

Have the model predict:

$$\sigma_v$$

or:

$$\log \sigma_v^2$$

Then:

$$R_{AI} = \sigma_v^2$$

becomes the measurement covariance.

This is one of the most important components I would preserve for the final SIH version.

TLIO is a strong conceptual reference here because it combines learned inertial motion and uncertainty with an EKF. [PubMed Central \(PMC\)](#)

33. Data Augmentation

This is essential.

Randomize:

Accelerometer bias

$$a' = a + b$$

Gyro bias

$$\omega' = \omega + b_g$$

Gaussian noise

$$a' = a + n$$

Scale

$$a' = S a$$

Rotation

$$a' = R a$$

Sampling jitter

Randomize timestamps slightly.

Sensor dropout

Remove occasional samples.

Vibration

Add high-frequency perturbations.

34. Why random rotation matters

You want:

Phone A:



Phone B:



Phone C:



to represent the same physical vehicle movement.

This is essential because the phone may be:

- horizontal
 - tilted
 - upside-down
 - slightly rotated in the holder.
-

35. Domain Randomization

Simulate multiple phone types:

noise:

low / medium / high

bias:

low / medium / high

sampling:

50 / 100 Hz

orientation:

random

This makes the model learn:

vehicle dynamics

rather than:

characteristics of one phone.

36. Train/Test Split

Do **not** randomly split windows.

Bad:

same trip:

window 1 → train

window 2 → test

Good:

Trip A/B/C → train

Trip D → validation

Trip E/F → test

Even better:

vehicle/driver/trip separation

This tests real generalization.

37. Training Experiments

Create a simple experiment matrix.

Experiment	Model	Physics loss	Augmentation
E0	MLP	No	No
E1	CNN	No	No
E2	GRU	No	Yes
E3	TCN	No	Yes
E4	TCN	Yes	Yes
E5	TCN residual	Yes	Yes
E6	TCN multi-task	Yes	Yes

You don't need 50 experiments.

Six good experiments are enough.

38. Training Procedure

For each experiment:

1. Load train windows
 2. Normalize using TRAIN statistics only
 3. Apply augmentation
 4. Train model
 5. Validate
 6. Early stopping
 7. Save best checkpoint
 8. Test ONCE
 9. Record:
 - velocity MAE
 - velocity RMSE
 - parameters
 - model size
 - inference time
-

39. Normalization

Store:

mean
std

for every input channel.

Then:

$$x' = \frac{x - \mu}{\sigma}$$

Save those values with the model.

Deployment must use exactly the same preprocessing.

40. Model Selection Criterion

Don't choose based solely on validation error.

Use:

$$\text{Score} = \text{Accuracy} - \lambda_1 \text{Latency} - \lambda_2 \text{ModelSize}$$

The best model is:

smallest model that gives nearly the best navigation performance.

41. Quantization

After selecting the model:

FP32

↓

FP16

↓

INT8

Benchmark all three.

Target:

<1–2 MB

if possible.

But don't artificially force it below that if accuracy suffers.

42. Export

Preferred deployment path:

PyTorch

↓

ONNX

↓

mobile runtime

or:

TensorFlow

↓

TensorFlow Lite

TensorFlow Lite is explicitly designed for small on-device models and supports Android and embedded Linux; its runtime can use CPU and available hardware acceleration. [GitHub](#)

ONNX Runtime is another strong option because its mobile runtime supports Android/iOS and browser inference through the broader ONNX Runtime ecosystem. [ONNX Runtime](#)

For your **Chrome web app**, I would lean toward **ONNX Runtime Web** because you want the same model to run in the browser.

43. The Core Navigation Filter

Use an EKF first.

State:

$$\mathbf{x} = [p_x, p_y, v_x, v_y, \psi, b_g, b_{ax}, b_{ay}]^T$$

Don't start with a monstrous 3D state.

For the first road-navigation prototype, a **2D vehicle model** is sufficient.

Later:

$$\mathbf{x}, \mathbf{y}, \mathbf{z}$$

can be added.

44. Prediction

Using IMU:

$$\psi_{k+1} = \psi_k + (\omega_z - b_g) \Delta t$$

and:

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t$$

The AI velocity becomes a measurement rather than replacing the filter.

45. AI Measurement

Network produces:

$$\mathbf{z}_{AI} = \begin{bmatrix} v_f \end{bmatrix}$$

with:

$$\mathbf{R}_{AI} = [\sigma_v^2]$$

The EKF decides how much to trust it.

46. GNSS Measurement

When available:

$$\mathbf{z}_{GNSS} = [p_x, p_y, v_x, v_y]$$

with covariance derived from GNSS accuracy.

Don't simply hard-code:

GPS always trusted.

47. GNSS Quality State

Compute:

$$\text{GNSS QUALITY} \in [0,1]$$

based on:

- reported accuracy
- position jumps
- velocity consistency
- heading consistency
- innovation residual
- signal availability

Then:

quality high
→ low R_GNSS

quality low
→ high R_GNSS

48. GNSS Outage Detector

Use innovation:

$$\mathbf{r} = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}$$

and normalized innovation:

$$\mathbf{NIS} = \mathbf{r}^T \mathbf{S}^{-1} \mathbf{r}$$

If it remains anomalous:

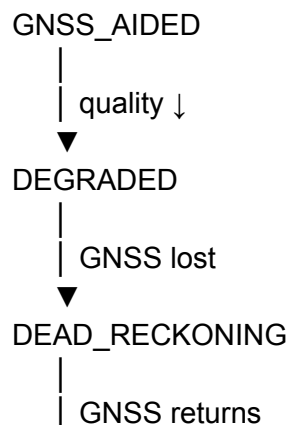
GNSS DEGRADED

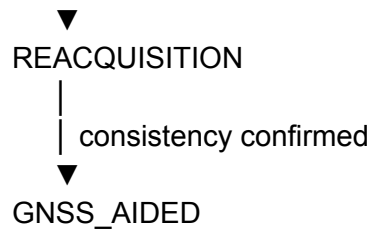
If unavailable:

GNSS DENIED

49. Navigation State Machine

Implement:





This should be a formal state machine, not scattered `if` statements.

50. NHC

Use:

$\|v_y\| \approx 0$

as a pseudo-measurement.

For ordinary road driving:

forward velocity $\rightarrow$ meaningful

lateral velocity $\rightarrow \sim 0$

This strongly limits lateral drift.

51. Simple Map Matching — Prototype

Don't build a GNN yet.

For each predicted position:

INS position



find roads within 30–50m



calculate:

distance

heading difference

road continuity



choose best road

Score:

$$S = w_d D + w_h H + w_t T$$

where:

- D : distance from road
- H : heading mismatch
- T : topology mismatch

Then use it as a **soft correction**.

52. HMM — Main SIH Version

Later:

candidate road segments



transition probabilities



emission probabilities



Viterbi



most probable road path

This is substantially safer than snapping to the nearest road.

53. GNN — Research Extension

Eventually represent the road network as:

$$G = (V, E)$$

Node:

intersection / road point

Edge:

road segment

GNN input:

candidate road

+

heading

+

distance

+

speed

+

previous road

Output:

road likelihood

But **do not make this part of V1**.

54. Why GNN is not the core model

Your data has two structures:

IMU

Temporal:

$[x_1, x_2, \dots, x_T]$

→ TCN.

Road network

Graph:

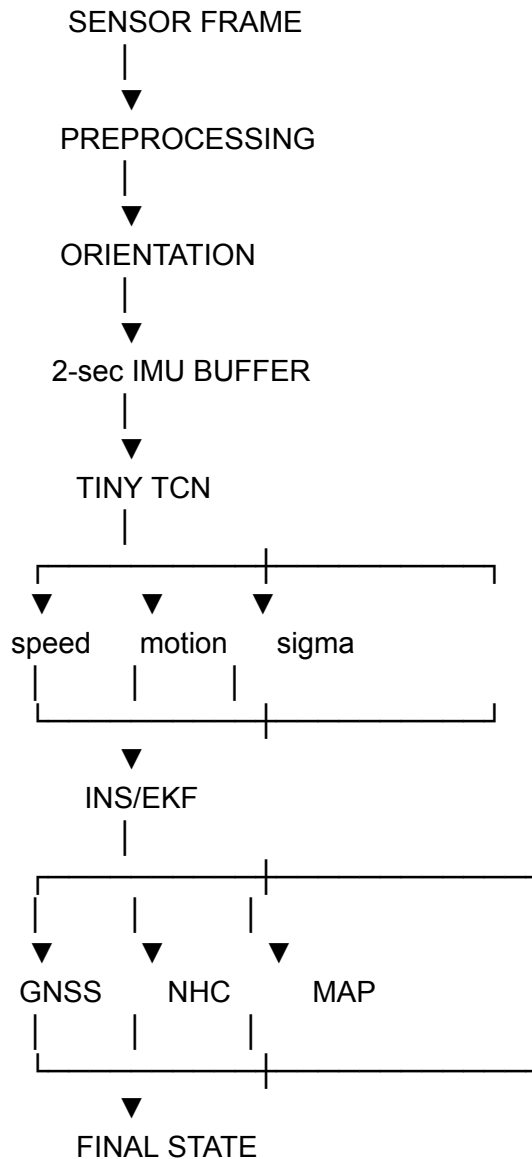
$[G(V, E)]$

→ GNN.

This division is clean and technically defensible.

55. Complete Core Pipeline

At runtime:



56. What the Core Engine Should Output

Create a single object:

```
{  
  "timestamp": 123456789,  
  "latitude": 26.1445,  
  "longitude": 91.7362,  
  "speed": 47.2,  
  "heading": 82.4,  
  "mode": "DEAD_RECKONING",  
  "gnssQuality": 0.0,  
  "aiConfidence": 0.94,  
  "mapConfidence": 0.91  
}
```

The UI team should consume **only this**.

They should not know about:

- Kalman matrices
- TCN tensors
- sensor bias
- map candidates.

This gives you a clean software boundary.

PART III — WEB APP + INTEGRATION

Now the interesting part.

You specifically want:

Mobile Chrome web application, no Google Maps API.

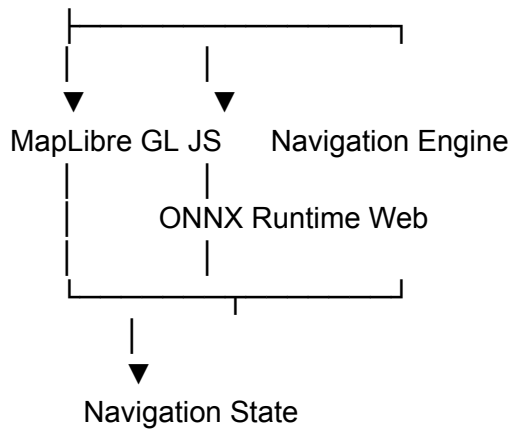
That is completely reasonable for a prototype.

57. Web App Architecture

I would use:

React / TypeScript

|



MapLibre GL JS is designed for interactive WebGL maps and supports vector-tile based maps, making it a strong fit for a custom mapping UI without Google Maps. [MapLibre](#)

58. Why MapLibre

You want:

- map rotation
- animated vehicle
- custom road styling
- vector layers
- trajectory rendering
- offline capability eventually

MapLibre is a good fit.

Alternative:

Leaflet

Much simpler.

For your flashy prototype:

MapLibre > Leaflet

because WebGL/vector rendering gives you more room for visual effects.

59. Map Data

Do not use Google Maps.

Use:

OpenStreetMap-derived data

For the prototype, you can pre-download the map around one specific location.

For example:

Guwahati

or whatever demonstration location you select.

The app loads:

```
maps/  
  guwahati/  
    roads.geojson  
    map.mbtiles  
    metadata.json
```

60. Important distinction

You have two map requirements:

Visualization

MapLibre + tiles.

Navigation constraint

Road graph.

These should be separate.

MapLibre

→ rendering

Road Graph

→ navigation algorithm

Don't force the renderer to become the navigation engine.

61. Prototype Map

For internal hackathon:

Use a pre-generated:

roads.geojson

containing:

- road centerlines
- road IDs
- connectivity
- heading
- road type

The navigation engine can use this directly.

62. Loading a Specific Location

The app can have:

Select demo area

[Guwahati ▼]

LOAD

Then:

/public/maps/guwahati/

is loaded locally.

No API call is required.

63. Better Final Architecture

Eventually package:

```
offline-map/  
|  
├── tiles/  
├── style.json  
├── roadgraph.bin  
└── metadata.json
```

The app can load the entire local navigation region.

64. PWA

Make the web application a:

Progressive Web App

so users can:

Chrome



Add to Home Screen



IDR app

This makes it feel like a mobile application without requiring an APK for the prototype.

65. Web App Pages

Keep it simple.

Page 1

Navigation

Main map.

Page 2

System

Sensor/AI diagnostics.

Page 3

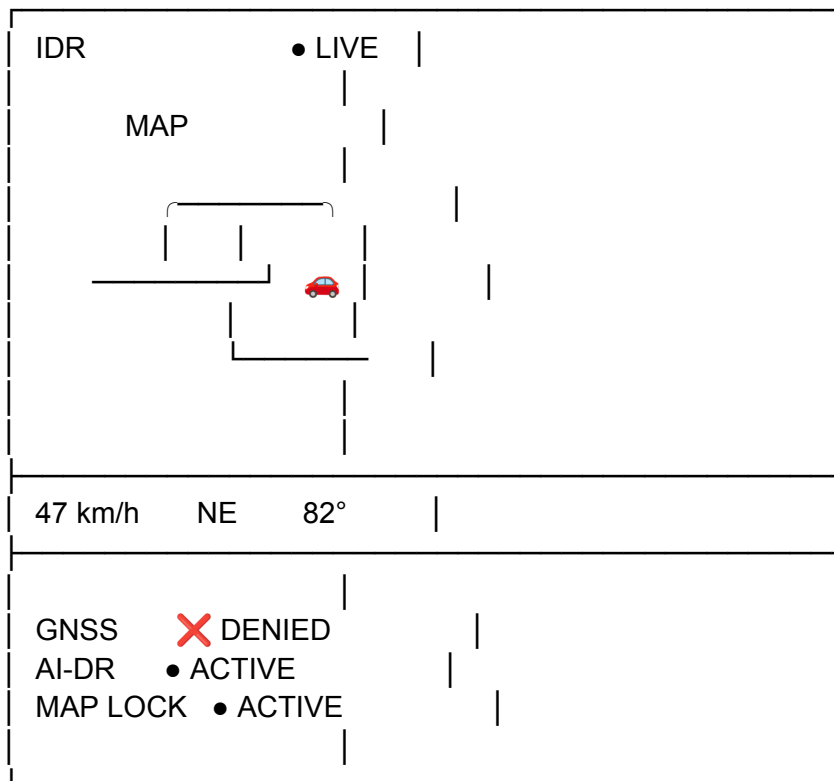
Demo

GNSS blackout controls.

For the internal hackathon, you might only need the first and third.

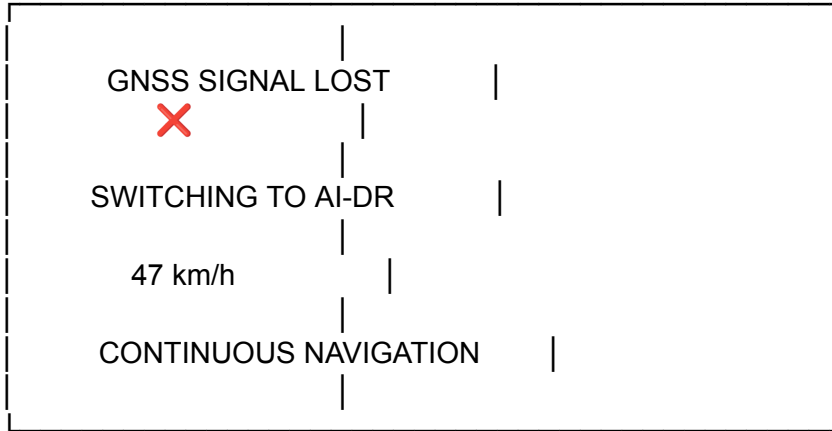
66. Main Navigation UI

I would make the main screen:



67. The "Wow" Screen

When GNSS disappears:



Animation:

GNSS



GNSS



AI-DR



68. System Status

Give judges a diagnostics screen:

SENSOR STATUS

Accelerometer • 100 Hz
Gyroscope • 100 Hz
Magnetometer • 50 Hz
GNSS • LOST

AI ENGINE

Model	TCN-v3
Inference	4.2 ms
Confidence	94%

NAVIGATION

Mode	DEAD RECKONING
Map Lock	91%
NHC	ACTIVE

This makes the prototype look much more mature.

69. Browser AI Inference

For the Web App:

Sensor data
↓
JS/TS preprocessing
↓
2-second tensor
↓
ONNX Runtime Web
↓
TCN
↓
velocity

ONNX Runtime supports inference across mobile and web environments, making it attractive if you want the same exported model to serve multiple deployment targets. [ONNX Runtime](#)

70. But there's an important browser constraint

Don't assume Chrome will expose every raw IMU sensor exactly like Android native APIs.

For the prototype, your architecture should allow:

Live sensor mode

and:

Replay mode

Replay mode is essential.

71. Replay Mode

Store a recorded sensor stream:

demo_trip.json

containing:

timestamp

ax

ay

az

gx

gy

gz

gnss

Then:

Play

Pause

GNSS blackout

Resume

This guarantees your demo works even if browser sensor permissions/hardware behave unexpectedly.

72. This is crucial for a hackathon

Your system should have:

MODE A
LIVE PHONE SENSORS

MODE B
RECORDED SENSOR REPLAY

If Chrome refuses sensor access during the presentation:

hit Replay.

The judges see the exact same pipeline.

73. Sensor Permission Flow

Mobile Chrome will require user permission/secure-context handling for sensor/location functionality, and exact browser behavior varies by device/browser.

So build:

START NAVIGATION

↓

Enable Motion Sensors

↓

Enable Location

↓

Calibrate Phone

↓

READY

Don't silently request everything.

74. Browser → Model Data Contract

Create:

```
type IMUSample = {
```

```
timestamp: number;  
ax: number;  
ay: number;  
az: number;  
gx: number;  
gy: number;  
gz: number;  
};
```

Model receives:

Float32Array

with shape:

[1, T, C]

or whatever exact ONNX input shape you settle on.

75. Browser → Navigation Engine

Use a single function:

```
navigationEngine.update(sensorFrame)
```

returns:

NavigationState

This makes the frontend completely independent.

76. Visualization Loop

Do not render at the raw IMU rate.

Instead:

IMU	100 Hz
AI	10 Hz

Navigation 20–100 Hz
UI 30–60 FPS

The UI interpolates vehicle position between navigation updates.

This makes the icon visually smooth.

77. Vehicle Animation

Keep:

actual state

separate from:

render state

For example:

navigation:

lat = X

lon = Y

renderer:

interpolates $X \rightarrow Y$

This avoids a jerky icon.

78. Map Layers

Create layers:

BASE MAP

ROAD GRAPH

GROUND TRUTH

GNSS TRACK

RAW INS TRACK

AI-DR TRACK

CURRENT VEHICLE

Then allow toggling:

- ☒ Ground truth
- ☒ GNSS
- ☒ Raw INS
- ☒ AI-DR

This will be extremely useful for your presentation.

79. Flashy Comparison Mode

Create:

COMPARE

The map displays:

GNSS ONLY red
RAW INS orange
AI-INS blue
AI + MAP green

During blackout:

GNSS ————— X

RAW INS ————///

AI-INS —————

AI+MAP —————

That is your strongest visual demo.

80. Offline Map

For the final system, the web app should eventually use:

PWA

+
service worker
+
local map assets
+
local road graph
+
local ML model

So:

Internet OFF



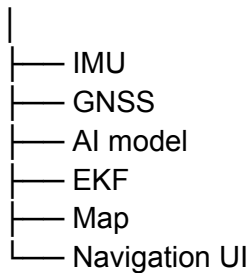
App still works

This is important because your use case is explicitly GNSS-denied environments.

81. No server dependency during navigation

The ideal final architecture is:

PHONE



No:

Phone → cloud → navigation

The PS explicitly wants edge/on-device execution.

82. Model Loading

Bundle:

```
/models/  
  idr_tcn.onnx  
  normalization.json
```

and:

```
/maps/  
  guwahati/
```

The browser loads the model once.

Don't reload it on every inference.

83. Navigation Engine Modules

Keep these independent:

```
core/  
|  
|— preprocessing.ts  
|— orientation.ts  
|— model.ts  
|— ins.ts  
|— ekf.ts  
|— nhc.ts  
|— gnss.ts  
|— mapmatch.ts  
|— stateMachine.ts  
|— navigation.ts
```

This modularity will pay off enormously when you later port the core to C++/Android/edge.

84. Testing Architecture

Create three test layers.

Unit

orientation
filter
EKF
coordinate conversion
map matching

Model

input → expected velocity

End-to-end

sensor replay
↓
AI
↓
EKF
↓
map
↓
web UI

The replay system makes end-to-end testing reproducible.

85. End-to-End Demo Data

Create a single canonical demonstration trip:

```
demo_trip/  
  sensors.csv  
  groundtruth.csv  
  map.geojson  
  metadata.json
```

Then define:

t = 0–30s:
GNSS ON

t = 30–90s:
GNSS OFF

t = 90–120s:
GNSS ON

The entire demo becomes deterministic.

86. Internal Hackathon Demo Sequence

0–10 seconds

Normal navigation.

GNSS •
AI-DR •

10–15 seconds

Vehicle enters tunnel.

GNSS ✗

15–40 seconds

AI-DR takes over.

AI-DR •
MAP LOCK •

40–45 seconds

GNSS returns.

GNSS REACQUIRED

45–60 seconds

Show comparison:

GNSS-only → failed
Raw INS → drifted

Our system → continued

Done.

87. Development Timeline

Sprint 1 — Data

Day 1

- download IO-VNBD
- parse one trip
- plot sensors
- understand coordinate systems

Day 2

- synchronization
- preprocessing
- ground truth velocity
- window generation

Day 3

- MLP
 - CNN baseline
-

88. Sprint 2 — AI

Day 4

GRU.

Day 5

TCN.

Day 6

TCN + augmentation.

Day 7

Multi-task heads.

89. Sprint 3 — Navigation

Day 8

Basic INS.

Day 9

EKF.

Day 10

AI velocity into EKF.

Day 11

NHC.

Day 12

GNSS outage state machine.

Day 13

Simple map matching.

Day 14

End-to-end replay.

90. Sprint 4 — Web

Parallel team:

React
+
MapLibre
+
PWA

Integrate:

NavigationState

into the UI.

91. Sprint 5 — Mobile

Export:

TCN → ONNX

Implement:

browser inference

Then:

live sensor → preprocessing → model

92. Sprint 6 — Polish

Add:

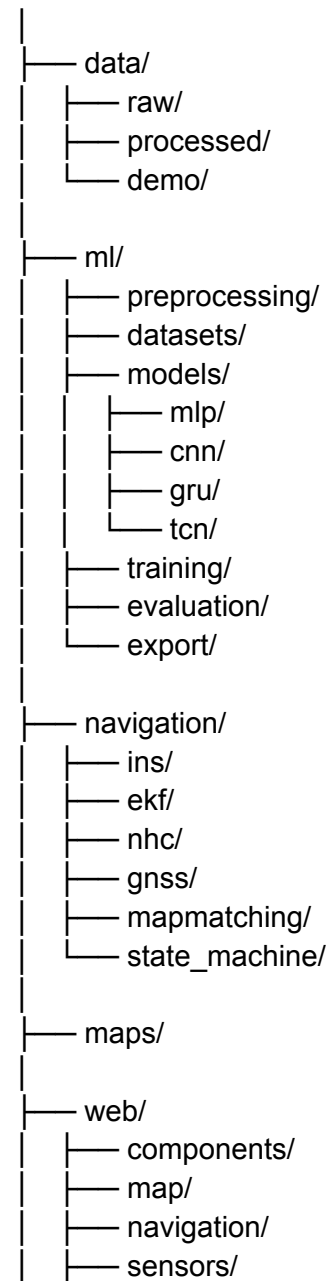
- comparison mode
- GNSS blackout animation
- confidence
- sensor diagnostics
- trajectory trails

- replay controls
 - offline map
-

93. Final Repository Structure

I recommend:

IDR/



```
|   └─ public/
|   └─ models/
|       └─ idr_tcn.onnx
|   └─ docs/
```

94. The three interfaces you must freeze early

This is probably the most useful software-engineering advice in the entire plan.

Interface 1 — Sensor → Core

SensorFrame

Interface 2 — Core → UI

NavigationState

Interface 3 — Model → Core

AIOutput

For example:

AIOutput:

velocity
lateralVelocity
motionClass
confidence
velocityVariance

If these three interfaces don't change, your team can work almost completely in parallel.

95. Parallel Team Structure

You can divide the team into:

Person/Team A — Sensor

phone → SensorFrame

You — ML + Navigation

SensorFrame

↓

AIOutput

↓

NavigationState

Person/Team C — Web

NavigationState

↓

Map/UI

Person/Team D — Dataset/Evaluation

IO-VNBD

↓

benchmarks

↓

plots

Integration

A + B + C

This is much cleaner than everyone working on the same code.

96. Exact MVP

If you need to cut the scope brutally:

Required

- ✓ phone IMU
- ✓ IO-VNBD
- ✓ preprocessing
- ✓ Tiny TCN

- ✓ speed estimation
- ✓ basic INS
- ✓ GNSS ON/OFF
- ✓ simple EKF
- ✓ NHC
- ✓ map projection
- ✓ web map
- ✓ live/replay

Not required yet

- X GNN
 - X full PINN
 - X camera
 - X FOG
 - X Raspberry Pi
 - X sophisticated HMM
 - X 3D navigation
 - X online learning
-

97. V2 — Main SIH Upgrade

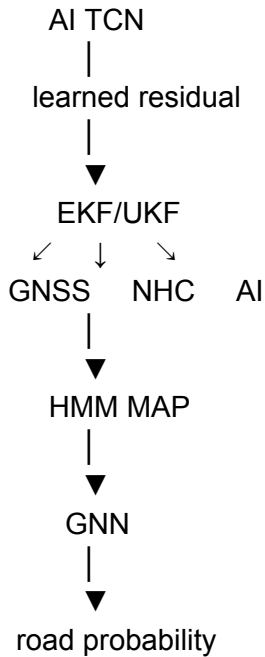
After internal hackathon:

MVP

- better TCN
 - residual learning
 - learned uncertainty
 - better alignment
 - HMM map matching
 - GNSS integrity
 - vehicle-type adaptation
 - domain randomization
 - real phone dataset
-

98. V3 — Advanced SIH

Then:



Optional:

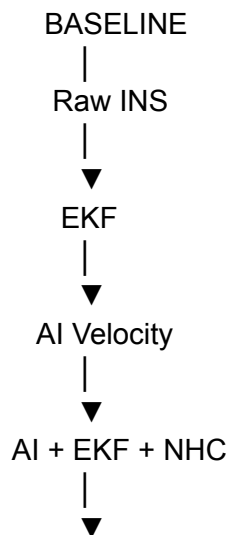
camera → visual odometry

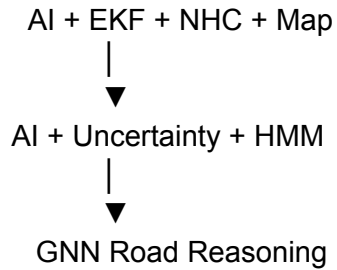
barometer → vertical constraint

FOG → edge engine

99. The final model hierarchy

I would explicitly document the evolution like this:





Every step is measurable.

100. What I would actually choose today

If you force me to make the technical decisions now:

Sensors

Accelerometer

Gyroscope

GNSS

Magnetometer only as auxiliary

Dataset

IO-VNBD

+

your own phone data later

Input

2-second rolling IMU window

Model

Tiny multi-task TCN

Outputs

forward velocity

lateral velocity

motion class

velocity uncertainty

Training

Huber velocity loss
+
physics consistency
+
NHC
+
smoothness
+
motion classification

Deployment

INT8 ONNX

Navigation

INS
+
EKF
+
AI velocity
+
adaptive covariance
+
NHC

Map

OpenStreetMap-derived offline road graph

Map matching

nearest-road + heading + topology

first, then HMM.

GNN

Future, not MVP.

UI

React + TypeScript
+
MapLibre
+
PWA

+
ONNX Runtime Web

Demo

Recorded replay + live sensor mode

101. One architectural decision I would insist on

Do not tie the ML model to latitude/longitude.

Your model should know nothing about:

Guwahati
Delhi
London
France

It should only understand:

IMU
→ vehicle motion

The navigation engine handles:

vehicle motion
→ trajectory

and the map engine handles:

trajectory
→ road

That separation is what eventually allows:

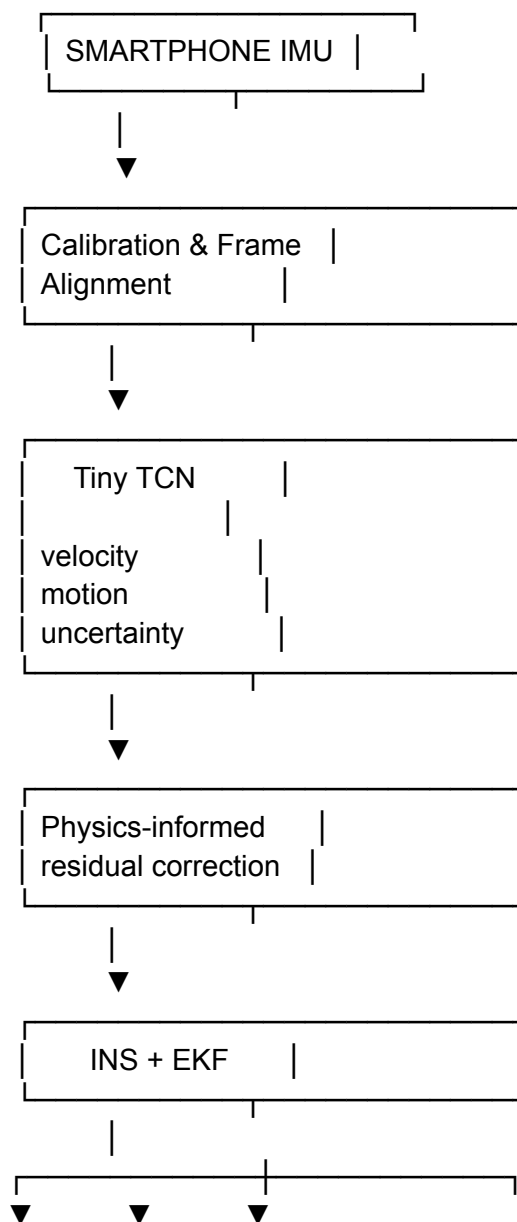
Smartphone MEMS
↓
same AI core
↓
Raspberry Pi
↓
external IMU

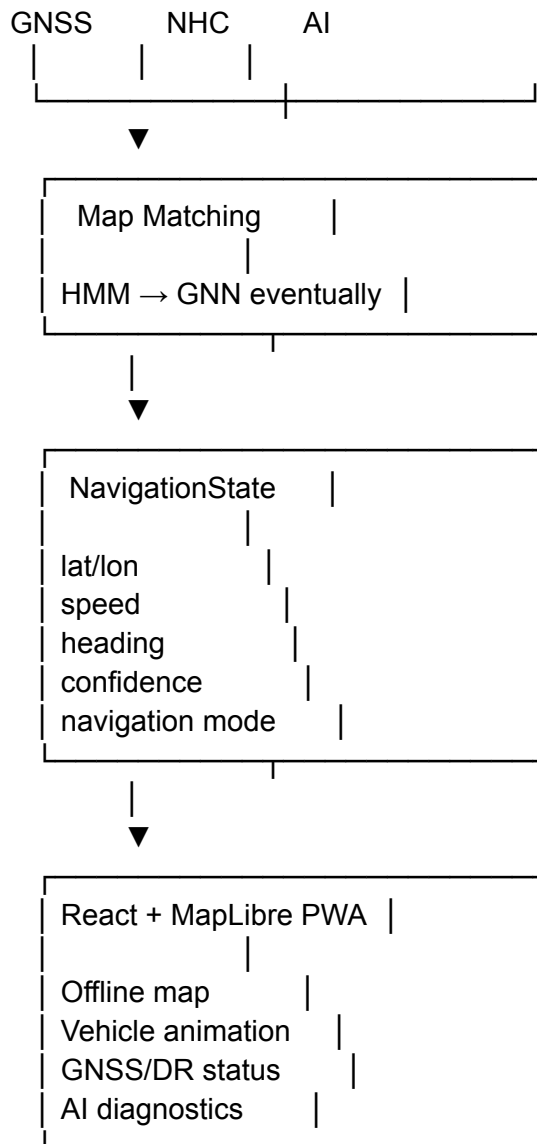
↓
FOG

without retraining the entire navigation architecture.

102. Final System

The final architecture you should work toward is:





The key point

For the **internal hackathon**, your actual target should be:

$$\boxed{\text{Phone IMU} \rightarrow \text{Tiny TCN} \rightarrow \text{AI velocity} \rightarrow \text{EKF + NHC} \rightarrow \text{Offline road} \rightarrow \text{Chrome PWA}}$$

and not the entire research-grade system.

The later additions—**physics-informed residual learning, adaptive uncertainty, HMM/GNN map reasoning, domain adaptation, camera/barometer, external IMU/FOG**—should all plug into this same architecture.

The nice part is that **IO-VNBD is large enough to get started immediately**, but its smartphone data are 10 Hz, so I would use it for the first model/benchmark and then collect a small high-rate dataset from your actual target phone. [ScienceDirect](#)

For the browser deployment, **ONNX Runtime Web + MapLibre + a PWA + prepackaged OSM-derived map data** gives you a clean no-Google-Maps architecture; ONNX Runtime supports mobile and browser inference, while MapLibre provides the interactive WebGL map layer. [ONNX Runtime](#)

The very next technical step I recommend is to freeze the *SensorFrame*, *AIOutput*, and *NavigationState* schemas, then build the IO-VNBD preprocessing pipeline. Once those three contracts are fixed, your sensor, ML, navigation, and UI people can work in parallel without blocking each other.